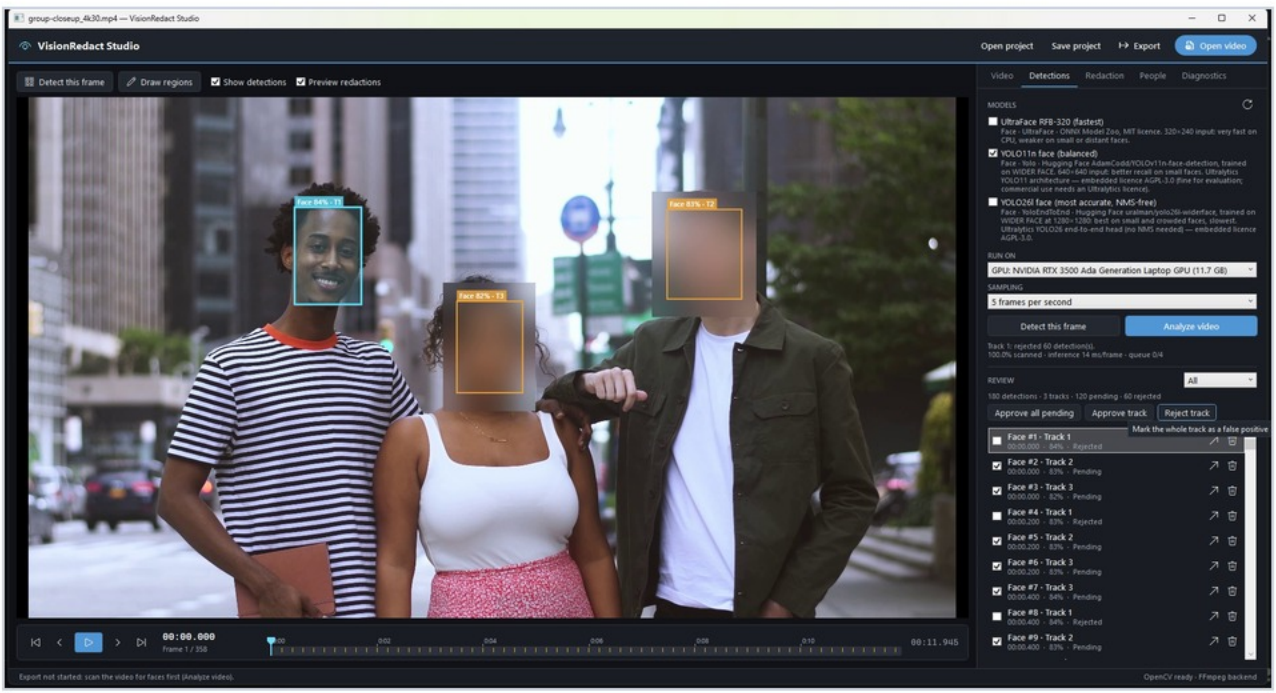
Redaction you can trust

A detected face stays blurred until someone rejects it. Fail-safe review, one planner for preview and export, and a parallel 4K export pipeline with FFmpeg.



C# .NET 10 OpenCV FFmpeg H.264

One rule for what gets hidden

AI boxes

every face starts as pending

↓

Your review

approve, reject, draw regions

↓

One planner

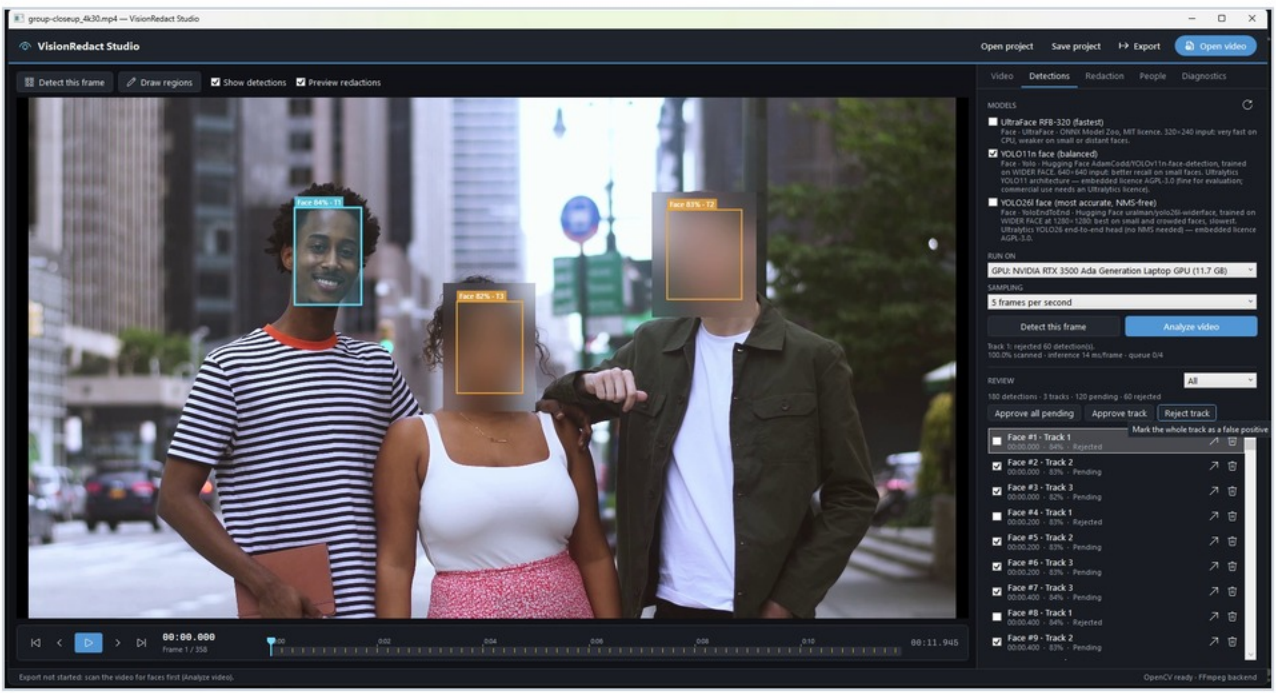
a pure function

↓

Preview = export

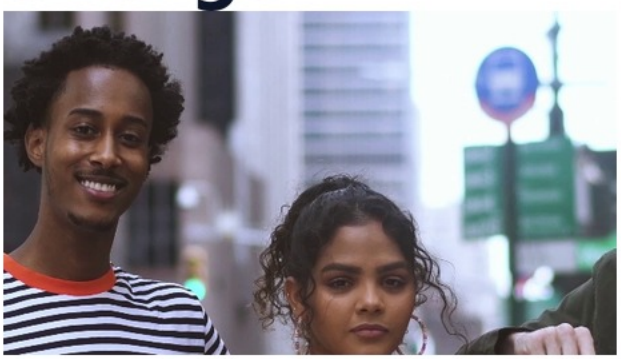
the same rectangles

A detected face stays blurred until someone rejects it

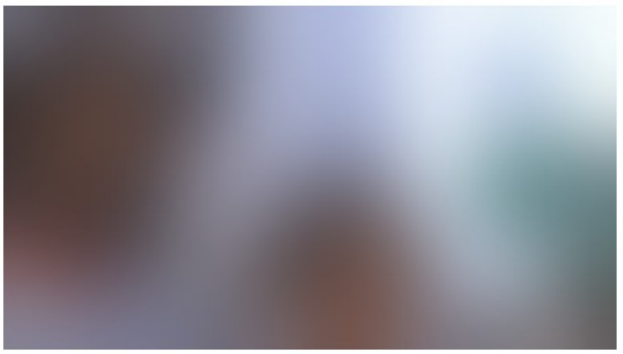


- Pending and Approved are blurred; Rejected is shown.
- In code it is one line: IsRedacted => Status != Rejected.
- Forget to review, and every face it found stays blurred.

Blur 41× faster: shrink, blur, enlarge



before



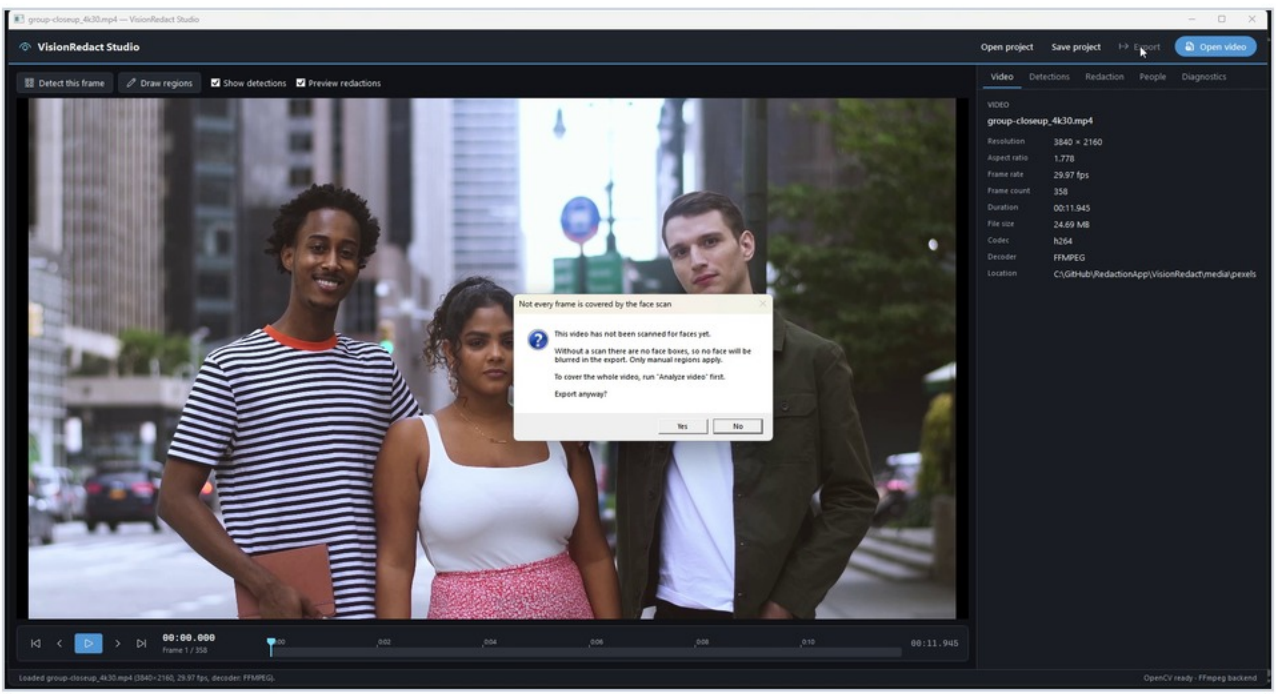
after: shrink, blur twice, enlarge

61.9 → 1.5 ms

per 576×324 region; export with a blurred region went from 13 to 75 frames per second.

Not bit-identical to one wide Gaussian; it looks the same at this strength. Solid black is the only effect that removes all information.

An unfinished scan? The app warns first



- Export checks the face-scan coverage first.
- It says which stretches were never scanned.
- No is the default. 99.96 % shows as 99.9 %, never 100 %.

What each check proves

1. **Coverage complete**

   every frame scanned or held; it can't show every face was found

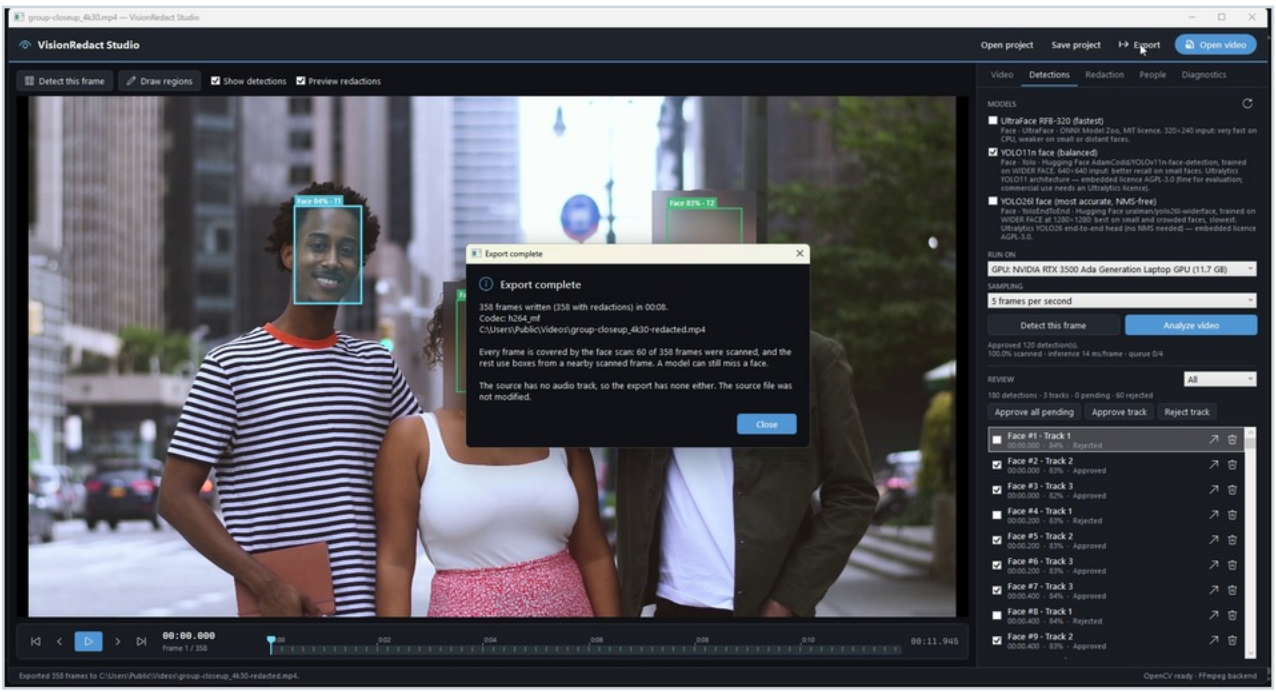
2. **Written = decoded**

   the encoder wrote every frame it got

3. **Decoded ≈ frame count**

   no large part of the source is missing; evidence, not proof

It says what it checked



- "Every frame is covered by the face scan … A model can still miss a face."
- Nothing half-written under the target name; the source is never modified.

Trade-offs of review and redaction

- A model can still miss a face: review and drawn regions are the backstop.
- The source check is evidence, not proof (an FFmpeg decoder would report errors).
- The fast blur is not bit-identical; solid black removes everything.

Export: three stages, in parallel

Decode

every frame, in order



Redact

the preview's planner and effects



Encode

FFmpeg: H.264 and the audio



Check, rename

only a complete file gets the name

Export speed, measured

1,800 frames of a 1080p59.94 camcorder file, one blurred region on every frame.

One loop



52 frames/s

Three stages



74 frames/s

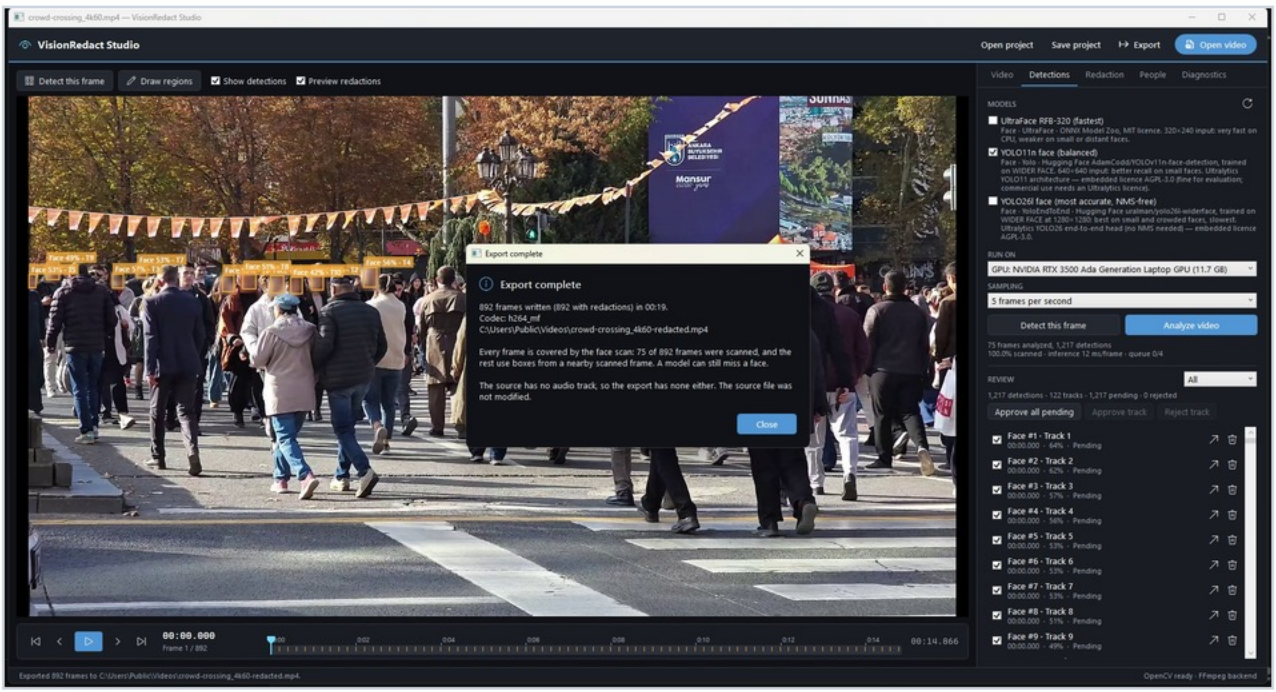
+ faster encoder



147–167 frames/s

Microsoft's H.264 encoder through FFmpeg: 147–167 frames/s over several runs, with audio (OpenCV's writer, OpenH264 only: 72–80, no audio). The 23-minute file: ≈26 → ≈9 minutes, computed from these rates.

4K60, every frame redacted



- 892 frames in 19 s on this laptop (1080p60: 5 s).
- The dialog says what was checked, and what wasn't.

Only a complete file gets the name

1. **Temporary file**
   frames go to <name>.partial-<guid>.mp4

2. **Count check**
   frames written (FFmpeg's count) = frames decoded

3. **Rename**
   only then does the file get its real name

4. **Cancel or fail**
   the temporary file is deleted

The audio bug

A 1,800-frame test export waited for FFmpeg to read the source's whole 23-minute audio track.

Before



49 frames/s

apad + -shortest



167 frames/s

The audio now ends with the video, also for full exports whose audio runs longer or shorter.

Half of a 4K decode is colour conversion

1 **Decode**

13.3 ms a 4K frame, 6.7 ms of it YUV → BGR

2 **Encode**

FFmpeg converts BGR back to NV12 (YUV)

3 **Next**

keep frames in YUV, or on the GPU: not measured yet

Trade-offs, and what's next

- An external process: about 180 MB of FFmpeg to ship.
- The software encoder uses CPU cores a hardware encoder would leave free.
- 4K60 exports run at 0.8× real time; the next bottleneck is not profiled yet.

Next episode

The engineering behind it: architecture, threads, and measurements that lied.

Where it's going: the whole pipeline on the GPU.